REACT

Objectives

- Define SPA and its benefits
- Define React and identify its working
- Identify the differences between SPA and MPA
- Explain Pros & Cons of Single-Page Application
- Explain about React
- Define virtual DOM
- Explain Features of React

In this hands-on lab, you will learn how to:

- Set up a react environment
- Use create-react-app

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes.**

Create a new React Application with the name “myfirstreact”, Run the application to print “welcome to the first session of React” as heading of that page.

1. To create a new React app, Install Nodejs and Npm from the following link:

   https://nodejs.org/en/download/

2. Install Create-react-app by running the following command in the command prompt:

   ```
   C:>npm install -g create-react-app
   ```

3. To create a React Application with the name of “myfirstreact”, type the following command:

   ```
   C:>npx create-react-app myfirstreact
   ```

4. Once the App is created, navigate into the folder of myfirstreact by typing the following command:

```
C:>cd myfirstreact
```

5. Open the folder of myfirstreact in Visual Studio Code
6. Open the App.js file in Src Folder of myfirstreact
7. Remove the current content of “App.js”
8. Replace it with the following:

```
function App() {
  return (
    <h1> Welcome the first session of React </h1>
  );
}
```

9. Run the following command to execute the React application:

```
C:\myfirstreact>npm start
```

10. Open a new browser window and type “localhost:3000” in the address bar

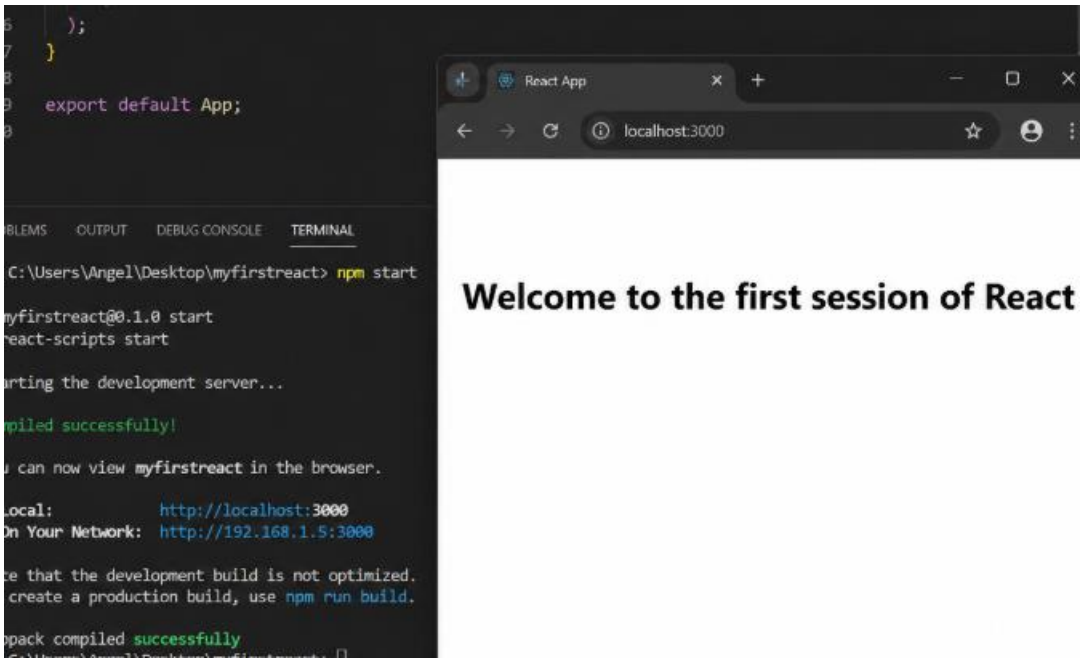
SOLUTION:

```
function App() {

  return (
    <div>
      <h1>Welcome to the first session of React</h1>
    </div>
  );
}
export default App;
```

RUN:

npm start

RESULT:



Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class component
- Explain function component
- Define component constructor
- Define render() function

In this hands-on lab, you will learn how to:

- Create a class component
- Create multiple components
- Render a component

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes.**

Create a react app for Student Management Portal named StudentApp and create a component named Home which will display the Message “Welcome to the Home page of Student Management Portal”. Create another component named About and display the Message “Welcome to the About page of the Student Management Portal”. Create a third component named Contact and display the Message “Welcome to the Contact page of the Student Management Portal”. Call all the three components.

1. Create a React project named “StudentApp” type the following command in terminal of Visual studio:

   ```
   C:>npx create-react-app StudentApp
   ```

2. Create a new folder under Src folder with the name “Components”. Add a new file named “Home.js”

3. Type the following code in Home.js

```
import React, {Component} from 'react';

import class Home extends Component{
   render(){
      <div>
      <h3> Welcome to the Home Page of Student Management Portal </h3>
      </div>
   }
}
```

4. Under Src folder add another file named “About.js”

5. Repeat the same steps for Creating “About” and “Contact” component by adding a new file as ”About.js”, “Contact.js” under “Src” folder and edit the code as mentioned for “Home” Component.

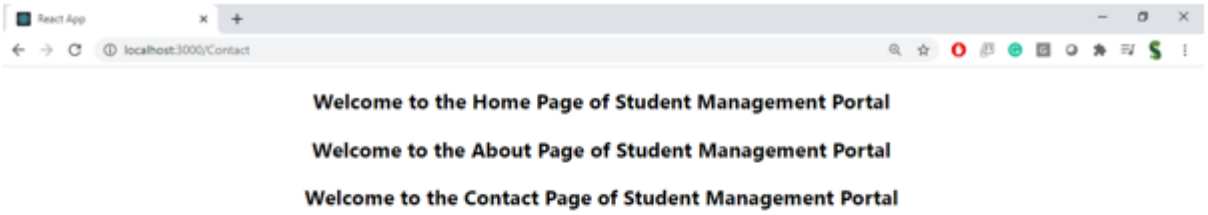
6. Edit the App.js to invoke the Home, About and Contact component as follows:

```
import logo from './logo.svg';
import './App.css';
import {Home} from './Components/Home';
import {About} from './Components/About';
import {Contact} from './Components/Contact';

function App() {
  return (
    <div className="container">
     <Home/>
     <About/>
     <Contact/>
    </div>
  );
}

export default App;
```

7. In command Prompt, navigate into StudentApp and execute the code by typing the following command:

```
C:\studentapp>npm start
```

8. Open browser and type “localhost:3000” in the address bar:



SOLUTION:

Home.js

src/Components/Home.js

```
import React, { Component } from "react";
class Home extends Component {
  render() {
    return (
      <div>
        <h2>Welcome to the Home page of Student Management Portal</h2>
      </div>
    );
  }
}
export default Home;
```

About.js

src/Components/About.js

```
import React, { Component } from "react";
class About extends Component {
  render() {
    return (
      <div>
        <h2>Welcome to the About page of the Student Management Portal</h2>
      </div>
    );
  }
}
export default About;
```

Contact.js

src/Components/Contact.js

```
import React, { Component } from "react";
class Contact extends Component {
  render() {
    return (
      <div>
        <h2>Welcome to the Contact page of the Student Management Portal</h2>
      </div>
    );
  }
}
export default Contact;
```

App.js

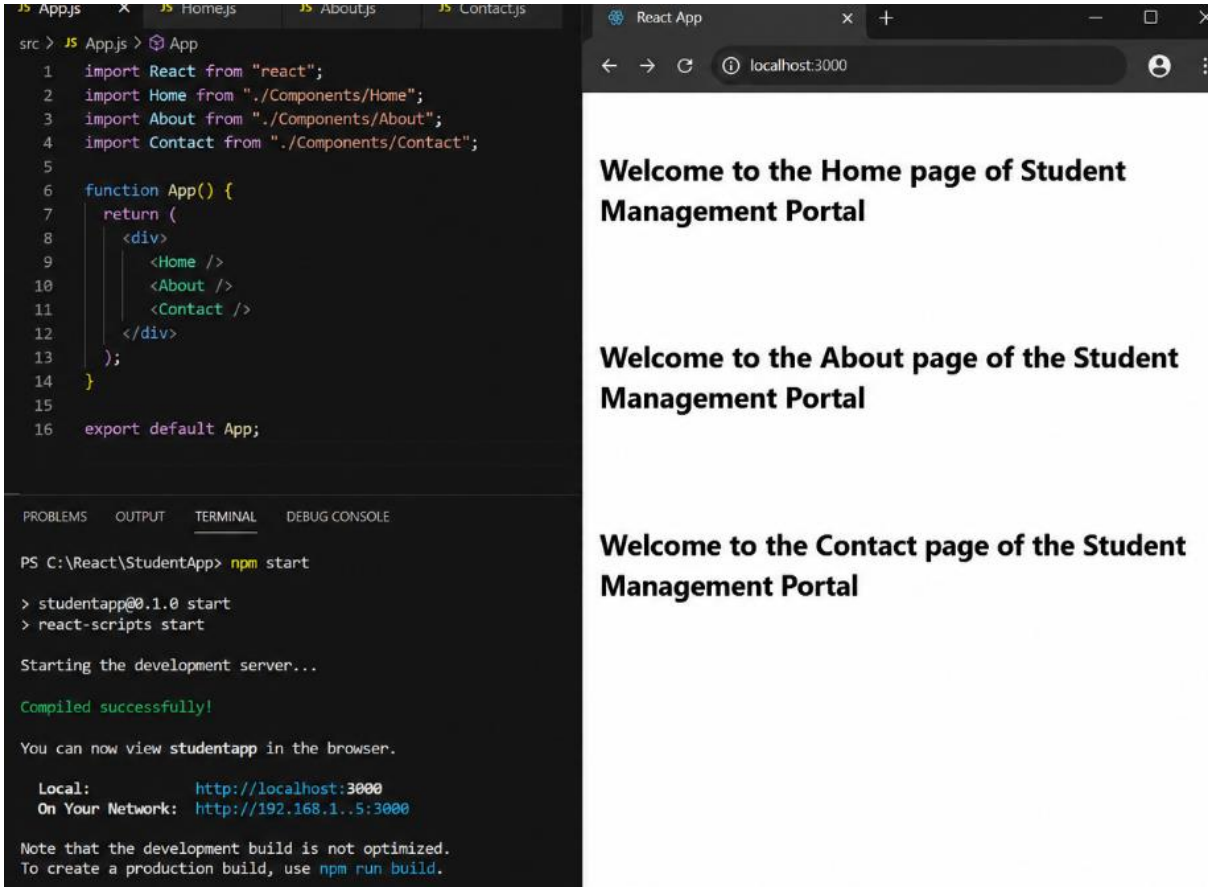
Replace the contents of src/App.js **with:**

```
import React from "react";
import Home from "./Components/Home";
import About from "./Components/About";
import Contact from "./Components/Contact";
function App() {
  return (
    <div>
      <Home />
      <About />
      <Contact />
    </div>
  );
}
export default App;
```

RUN:

npm start

RESULT:


App.js
Home.js
About.js
Contact.js
src > App.js > App
1 import React from "react";
2 import Home from "./Components/Home";
3 import About from "./Components/About";
4 import Contact from "./Components/Contact";
5
6 function App() {
7 return (
8
9 <Home />
10 <About />
11 <Contact />
12
13);
14 }
15
16 export default App;
PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
PS C:\React\StudentApp> npm start
> studentapp@0.1.0 start
> react-scripts start
Starting the development server...
Compiled successfully!
You can now view studentapp in the browser.
Local: http://localhost:3000
On Your Network: http://192.168.1..5:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
React App
localhost:3000
Welcome to the Home page of Student Management Portal
Welcome to the About page of the Student Management Portal
Welcome to the Contact page of the Student Management Portal

Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class component
- Explain function component
- Define component constructor
- Define render() function

In this hands-on lab, you will learn how to:

- Create a function component
- Apply style to components
- Render a component

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes.**

Create a react app for Student Management Portal named scorecalculatorapp and create a function component named “CalculateScore” which will accept Name, School, Total and goal in order to calculate the average score of a student and display the same.

9. Create a React project named “scorecalculatorapp” type the following command in terminal of Visual studio:

   ```
   C:>npx create-react-app scorecalculatorapp
   ```

10. Create a new folder under Src folder with the name “Components”. Add a new file named “CalculateScore.js”

11. Type the following code in CalculateScore.js

```
import '../Stylesheets/mystyle.css'

const percentToDecimal= (decimal) => {
   return (decimal.toFixed(2) + '%')
}

const calcScore = (total, goal) => {
   return percentToDecimal(total/goal)
}
```

```
export const CalculateScore = ({Name,School, total, goal}) => (
      <div className="formatstyle">
         <h1><font color="Brown">Student Details:</font></h1>
         <div className="Name">
           <b> <span> Name:  </span> </b>
           <span>{Name}</span>
         </div>
         <div className="School">
           <b> <span> School: </span> </b>
           <span>{School}</span>
         </div>
         <div className="Total">
           <b><span>Total:</span> </b>
           <span>{total}</span>
           <span>Marks</span>
         </div>
         <div className="Score">
           <b>Score:</b>
           <span>
             {calcScore(
               total,
               goal
             )}
           </span>
         </div>
      </div>
```

12. Create a Folder named Stylesheets and add a file named “mystyle.css” in order to add some styles to the components:

```
.Name
{
  font-weight:300;
 color:blue;
}
.School
{
 color:crimson;
}
.Total
{
 color:darkmagenta;
}
.formatstyle
{
   text-align:center;
   font-size:large;
}
.Score
{
   color:forestgreen;
}
```

13. Edit the App.js to invoke the CalculateScore functional component as follows:

```
import {CalculateScore} from '../src/components/CalculateScore';

function App()
{

return(
<div>
   <CalculateScore Name={"Steeve"}
   School={"DNV Public School"}
   total={284}
   goal={3}
   />
</div>
)
}

export default App;
```

14. In command Prompt, navigate into scorecalculatorapp and execute the code by typing the following command:

```
C:\scorecalculatorapp>npm start
```

15. Open browser and type “localhost:3000” in the address bar:

SOLUTION:

CalculateScore.js

src/Components/CalculateScore.js

```
import React from "react";
import "../Stylesheets/mystyle.css";
function CalculateScore(props) {
   const average = props.total / props.goal;
   return (
      <div className="box">
         <h2>Student Management Portal</h2>
         <p><strong>Name :</strong> {props.name}</p>
         <p><strong>School :</strong> {props.school}</p>
         <p><strong>Total Marks :</strong> {props.total}</p>
         <p><strong>Number of Subjects :</strong> {props.goal}</p>
         <h3>Average Score : {average}</h3>
      </div>
   );
}
export default CalculateScore;
```

mystyle.css

src/Stylesheets/mystyle.css

```
body{
```

```
  background-color:#f2f2f2;
  font-family:Arial, Helvetica, sans-serif;
}
.box{
  width:420px;
  margin:40px auto;
  padding:20px;
  background:white;
  border-radius:10px;
  box-shadow:0 0 10px gray;
}
h2{
  color:darkblue;
  text-align:center;
}
h3{
  color:green;
}
```

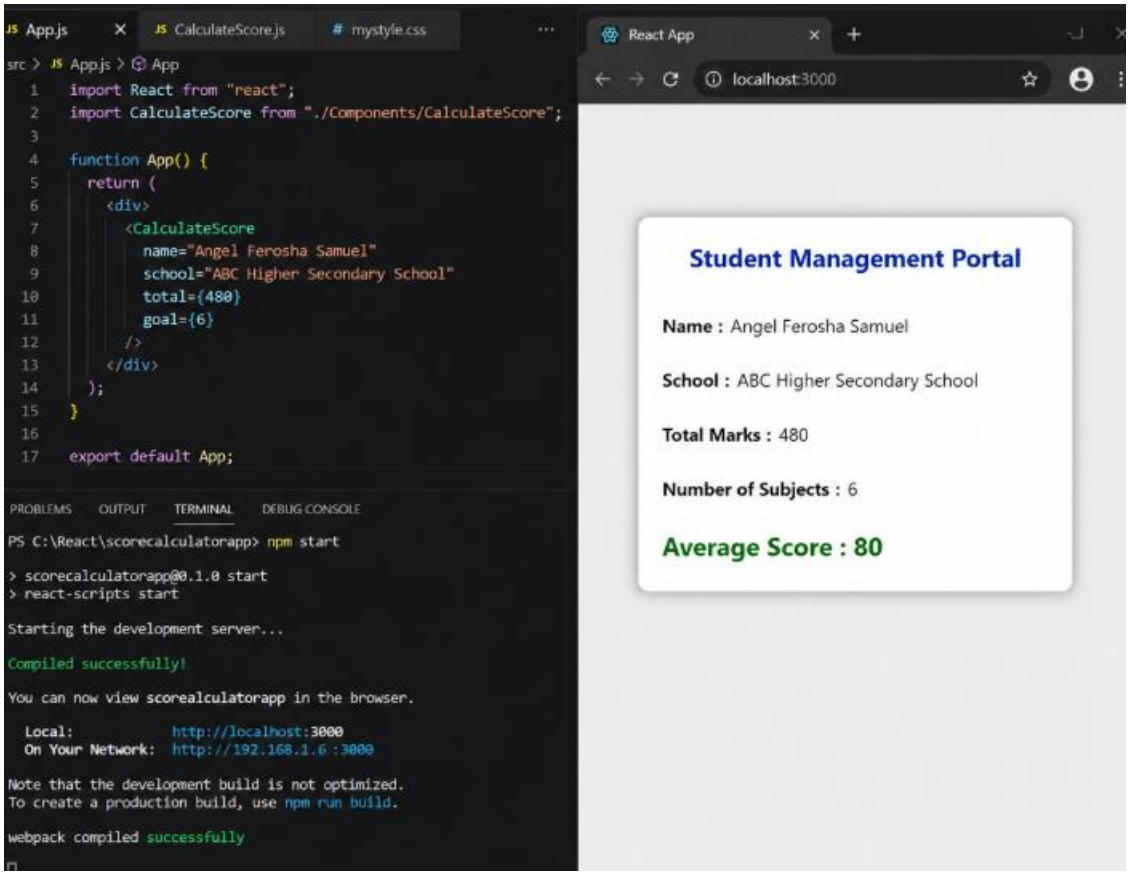
App.js

```
import React from "react";
import CalculateScore from "./Components/CalculateScore";
function App() {
  return (
    <div>
      <CalculateScore
        name="Angel Ferosha Samuel"
        school="ABC Higher Secondary School"
        total={480}
        goal={6}
      />
    </div>
  );
}
export default App;
```

RUN:

```
npm start
```

RESULT:


App.js
CalculateScore.js
mystyle.css
src > App.js > App
import React from "react";
import CalculateScore from "./Components/CalculateScore";
function App() {
return (

<CalculateScore
name="Angel Ferosha Samuel"
school="ABC Higher Secondary School"
total={480}
goal={6}
/>

);
}
export default App;
PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
PS C:\React\scorecalculatorapp> npm start
> scorecalculatorapp@0.1.0 start
> react-scripts start
Starting the development server...
Compiled successfully!
You can now view scorealculatorapp in the browser.
Local: http://localhost:3000
On Your Network: http://192.168.1.6:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully
React App
localhost:3000
Student Management Portal
Name : Angel Ferosha Samuel
School : ABC Higher Secondary School
Total Marks : 480
Number of Subjects : 6
Average Score : 80

Objectives

- Explain the need and Benefits of component life cycle
- Identify various life cycle hook methods
- List the sequence of steps in rendering a component

In this hands-on lab, you will learn how to:

- Implement componentDidMount() hook
- Implementing componentDidCatch() life cycle hook.

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **60 minutes.**

1. Create a new react application using *create-react-app* tool with the name as "blogapp"
2. Open the application using VS Code
3. Create a new file named as **Post.js** in **src folder** with following properties

```
Post.js  U
1   class Post {
2       constructor(id, title, body){
3           this.id=id;
4           this.title=title;
5           this.body=body;
6       }
7   }
8   export default Post;
```

Figure 1: Post class

4. Create a new class based component named as **Posts** inside **Posts.js** file

```
Posts.js U
1  class Posts extends React.Component {
2      constructor(props){
3          super(props);
4      }
5  }
```

Figure 2: Posts Component

5. Initialize the component with a list of Post in state of the component using the constructor
6. Create a new method in component with the name as **loadPosts()** which will be responsible for using Fetch API and assign it to the component state created earlier. To get the posts use the url (https://jsonplaceholder.typicode.com/posts)

```
Posts.js U
1  class Posts extends React.Component {
2      constructor(props){
3          super(props);
4          //code
5      }
6      loadPosts() {
7          //code
8      }
9  }
```

Figure 3: loadPosts() method

7. Implement the **componentDidMount()** hook to make calls to **loadPosts()** which will fetch the posts

```
Posts.js U
1  class Posts extends React.Component {
2      constructor(props){
3          super(props);
4          //code
5      }
6      loadPosts() {
7          //code
8      }
9      componentDidMount() {
10         //code
11     }
12 }
```

Figure 4: componentDidMount() hook

8. Implement the **render()** which will display the title and post of posts in html page using heading and paragraphs respectively.

```
JS Posts.js U
1   class Posts extends React.Component {
2       constructor(props) {…
5       }
6       loadPosts() {…
8       }
9       componentDidMount() {…
11      }
12      render() {
13          //code
14      }
15  }
```

Figure 5: render() method

9. Define a **componentDidCatch()** method which will be responsible for displaying any error happing in the component as alert messages.

```
JS Posts.js U
1   class Posts extends React.Component {
2       constructor(props) {…
5       }
6       loadPosts() {…
8       }
9       componentDidMount() {…
11      }
12      render() {…
14      }
15      componentDidCatch(error, info) {
16          //code
17      }
18  }
```

Figure 6: componentDidCatch() hook

10. Add the Posts component to App component.
11. Build and Run the application using *npm start* command.

SOLUTION:

Post.js

```
class Post {
  constructor(id, title, body) {
    this.id = id;
    this.title = title;
    this.body = body;
  }
```

```
}
export default Post;
```

Posts.js

```
import React, { Component } from "react";
import Post from "./Post";
class Posts extends Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: []
    };
  }
  loadPosts() {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(response => response.json())
      .then(data => {
        const posts = data.map(
          item => new Post(item.id, item.title, item.body)
        );
        this.setState({
          posts: posts
        });
      })
      .catch(error => {
        throw error;
      });
  }
  componentDidMount() {
    this.loadPosts();
  }
  componentDidCatch(error, info) {
    alert("An Error Occurred");
    console.log(error);
    console.log(info);
  }
  render() {
    return (
      <div>
        <h1>Blog Posts</h1>
        {
          this.state.posts.map(post => (
            <div key={post.id}>
              <h2>{post.title}</h2>
              <p>{post.body}</p>
              <hr />
            </div>
```

```
            ))
          }
        </div>
      );
  }
}
export default Posts;
```

App.js

```
import React from "react";
import Posts from "./Posts";
function App() {
  return (
    <div>
      <Posts />
    </div>
  );
}
export default App;
```

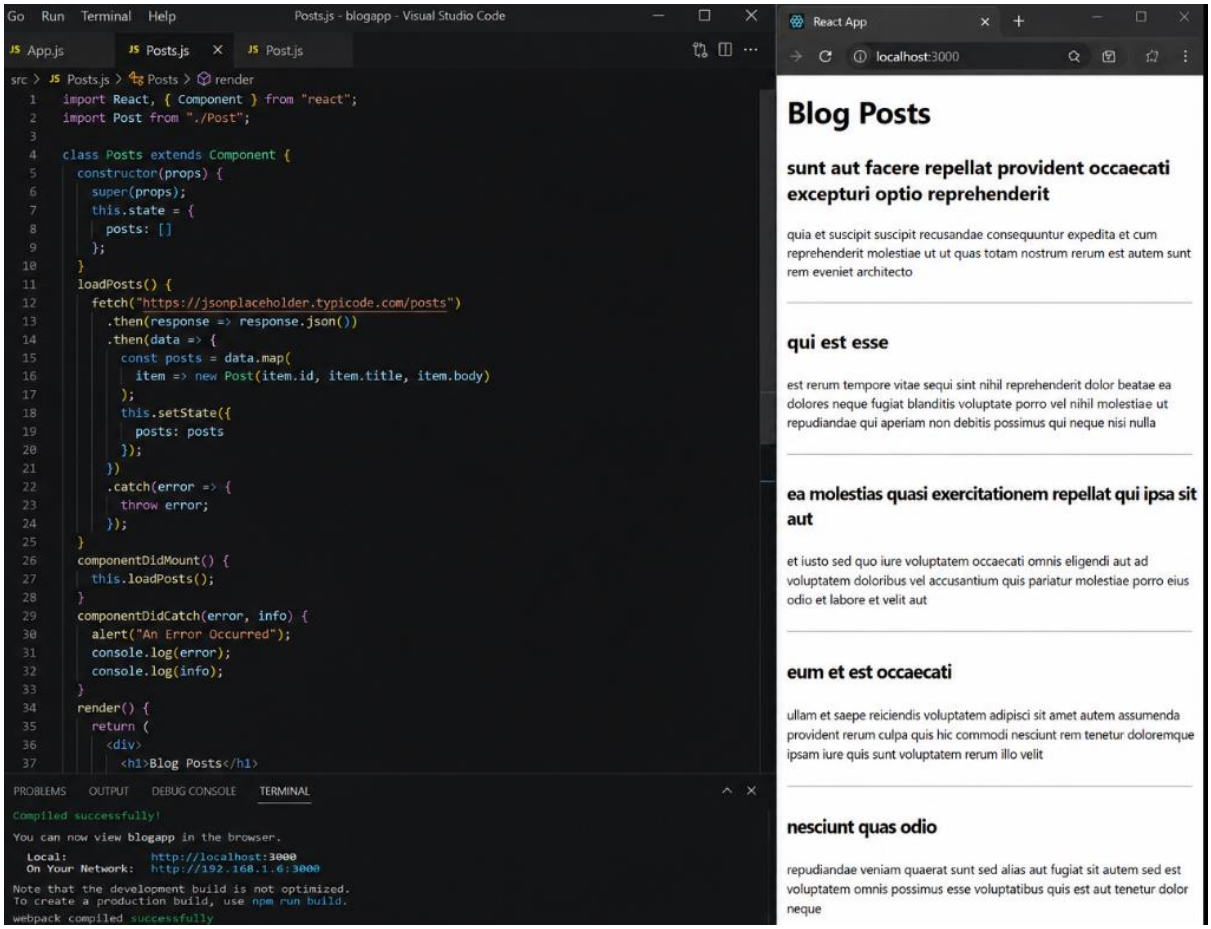
index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

RUN:

```
npm start
```

RESULT:


Go Run Terminal Help
Posts.js - blogapp - Visual Studio Code
App.js
Posts.js
Post.js
src > Posts.js > Posts > render
import React, { Component } from "react";
import Post from "./Post";
class Posts extends Component {
constructor(props) {
super(props);
this.state = {
posts: []
};
}
loadPosts() {
fetch("https://jsonplaceholder.typicode.com/posts")
.then(response => response.json())
.then(data => {
const posts = data.map(
item => new Post(item.id, item.title, item.body)
);
this.setState({
posts: posts
});
})
.catch(error => {
throw error;
});
}
componentDidMount() {
this.loadPosts();
}
componentDidCatch(error, info) {
alert("An Error Occurred");
console.log(error);
console.log(info);
}
render() {
return (

<h1>Blog Posts</h1>
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Compiled successfully!
You can now view blogapp in the browser.
Local: http://localhost:3000
On Your Network: http://192.168.1.6:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully
React App
localhost:3000
Blog Posts
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto
qui est esse
est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditis voluptate porro vel nihil molestiae ut repudiandae qui aperiam non debitis possimus qui neque nisi nulla
ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit
nesciunt quas odio
repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

Objectives

- Understanding the need for styling react component
- Working with CSS Module and inline styles

In this hands-on lab, you will learn how to:

- Style a react component
- Define styles using the CSS Module
- Apply styles to components using className and style properties

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes.**

My Academy team at Cognizant want to create a dashboard containing the details of ongoing and completed cohorts. A react application is created which displays the detail of the cohorts using react component. You are assigned the task of styling these react components.

Download and build the attached react application.

1. Unzip the react application in a folder
2. Open command prompt and switch to the react application folder
3. Restore the node packages using the following commands

Figure 7: Restore packages

4. Open the application using VS Code

5. Create a new CSS Module in a file called “CohortDetails.module.css”
6. Define a css class with the name as “box” with following properties

    Width = 300px;

    Display = inline block;

    Overall 10px margin

    Top and bottom padding as 10px

    Left and right padding as 20px

    1 px border in black color

    A border radius of 10px
7. Define a css style for html <dt> element using tag selector. Set the font weight to 500.
8. Open the cohort details component and import the CSS Module
9. Apply the box class to the container div
10. Define the style for <h3> element to use “green” color font when cohort status is “ongoing” and “blue” color in all other scenarios.
11. Final result should look similar to the below image

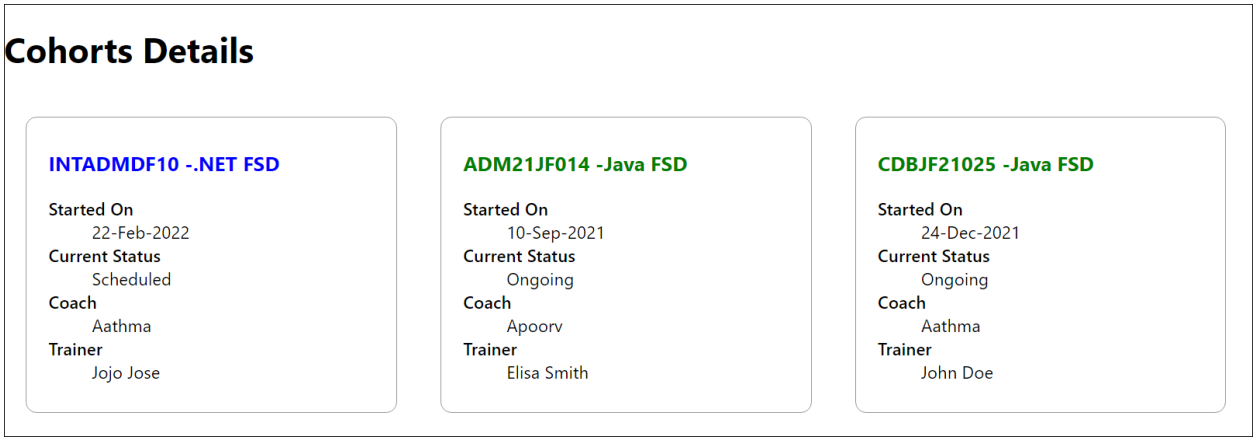


Figure 8: Final Result

SOLUTION:

CohortDetails.module.css

```
.box {
   width: 300px;
   display: inline-block;
   margin: 10px;
   padding: 10px 20px;
   border: 1px solid black;
   border-radius: 10px;
}
dt {
   font-weight: 500; }
```

CohortDetails.js

```
import React from "react";
import styles from "../CohortDetails.module.css";
function CohortDetails(props) {
  const headingStyle = {
    color: props.status === "ongoing" ? "green" : "blue"
  };
  return (
    <div className={styles.box}>
      <h3 style={headingStyle}>
        {props.cohortName}
      </h3>
      <dl>
        <dt>Started On</dt>
        <dd>{props.startDate}</dd>
        <dt>Current Status</dt>
        <dd>{props.status}</dd>
        <dt>Coach</dt>
        <dd>{props.coach}</dd>
        <dt>Trainer</dt>
        <dd>{props.trainer}</dd>
      </dl>
    </div>
  );
}
export default CohortDetails;
```

Example App.js

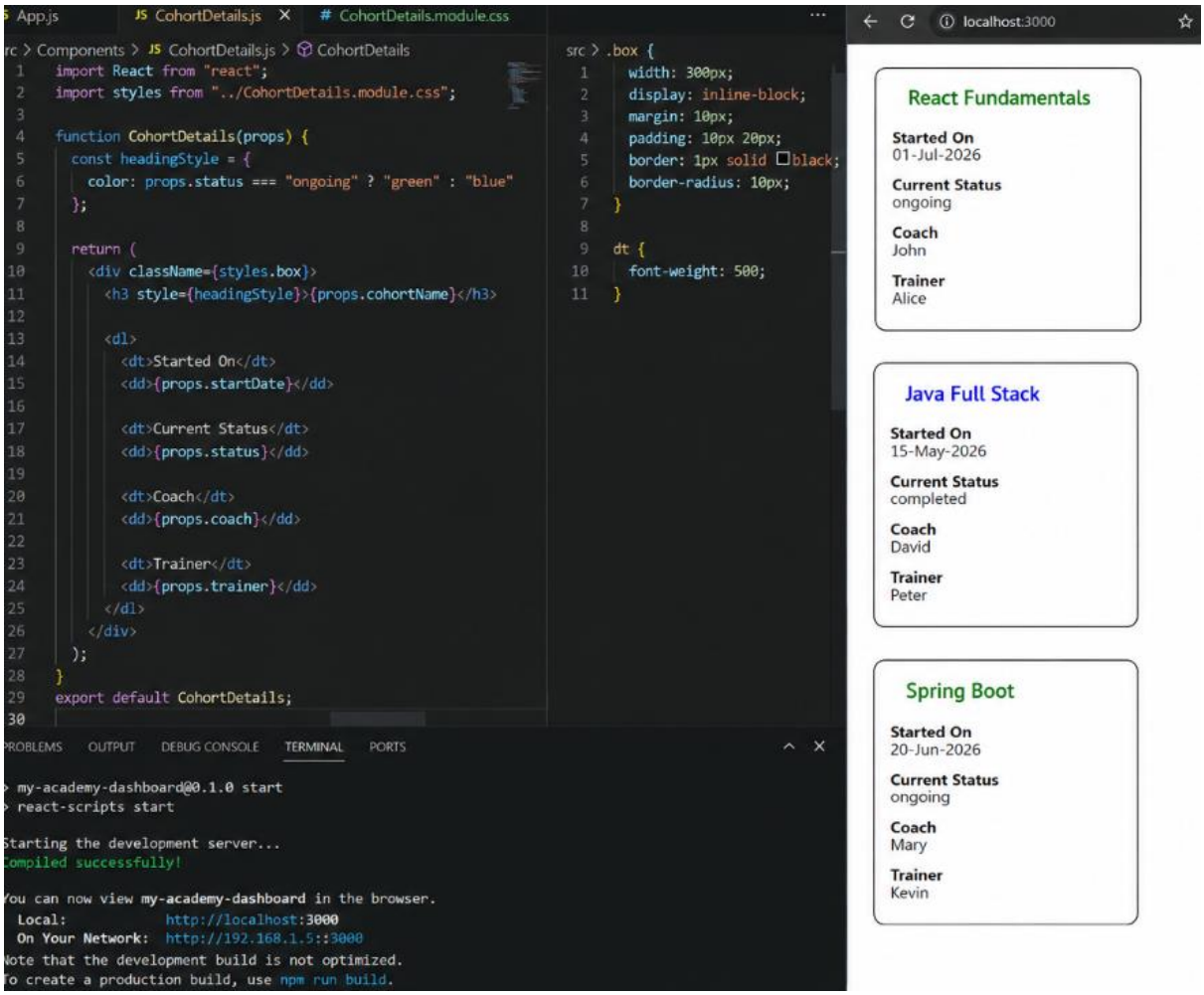
```
import React from "react";
import CohortDetails from "./Components/CohortDetails";
function App() {
  return (
    <div>
      <CohortDetails
        cohortName="React Fundamentals"
        startDate="01-Jul-2026"
        status="ongoing"
        coach="John"
        trainer="Alice"
      />
      <CohortDetails
        cohortName="Java Full Stack"
        startDate="15-May-2026"
        status="completed"
        coach="David"
        trainer="Peter"
      />
```

```
        <CohortDetails
          cohortName="Spring Boot"
          startDate="20-Jun-2026"
          status="ongoing"
          coach="Mary"
          trainer="Kevin"
        />
      </div>
    );
}
export default App;
```

RUN:

```
npm install
npm start
```

RESULT:



Objectives

- List the features of ES6
- Explain JavaScript let
- Identify the differences between var and let
- Explain JavaScript const
- Explain ES6 class fundamentals
- Explain ES6 class inheritance
- Define ES6 arrow functions
- Identify set(), map()

In this hands-on lab, you will learn how to:

- Use map() method of ES6
- Apply arrow functions of ES6
- Implement Destructuring features of ES6

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **60 minutes.**

Create a React Application named “cricketapp” with the following components:

1. ListofPlayers
   - Declare an array with 11 players and store details of their names and scores using the map feature of ES6

```
return(
      players.map((item)=>
      {
      return(
         <div>
            <li>Mr. {item.name}<span> {item.score} </span></li>
         </div>

      )})
      )
```

   - Filter the players with scores below 70 using arrow functions of ES6.

```
return(
      players.map((item)=>
     {
       if(item.score<=70)
         {
             players70.push(item);
         }
       }),
```

2. IndianPlayers

    a. Display the Odd Team Player and Even Team players using the Destructuring features of ES6

```
export function OddPlayers([first,, third,,fifth]) {
   return(
   <div>
      <li> First :  {first}  </li>
      <li> Third :  {third} </li>
      <li> Fifth : {fifth}</li>
   </div>)}
```

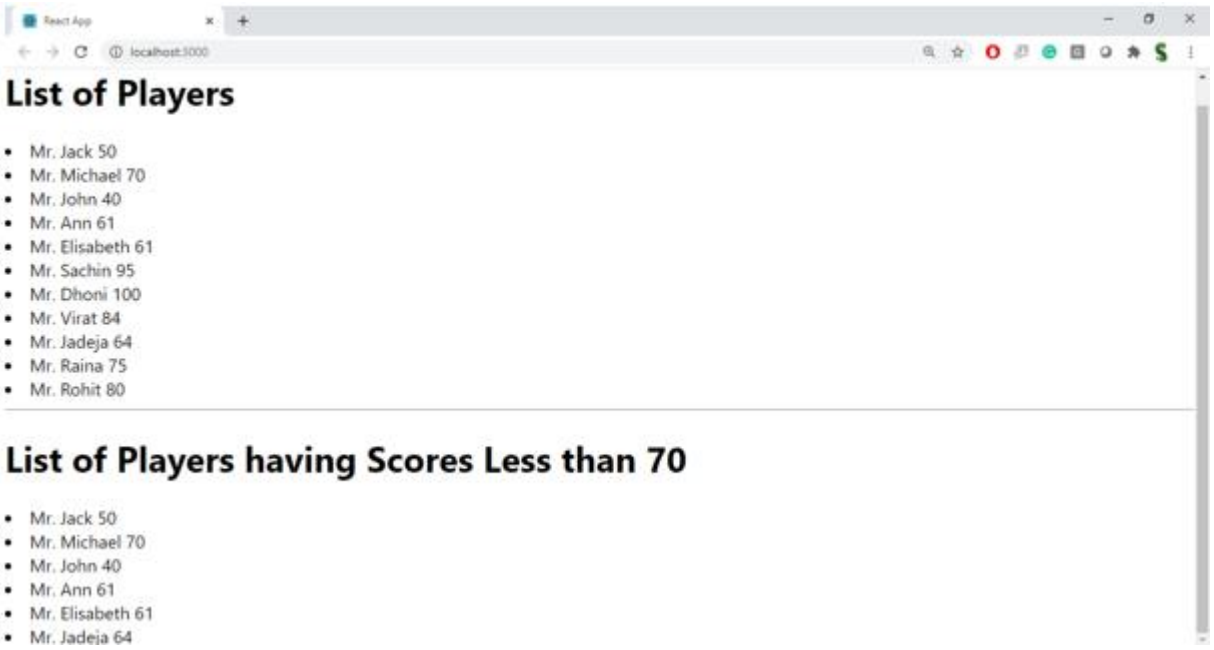
    b. Declare two arrays T20players and RanjiTrophy players and merge the two arrays and display them using the Merge feature of ES6

```
const T20Players=['First Player','Second Player','Third Player'];
   const RanjiTrophyPlayers=['Fourth Player','Fifth Player','Sixth Player'];
  export const IndianPlayers=[...T20Players, ...RanjiTrophyPlayers]
```

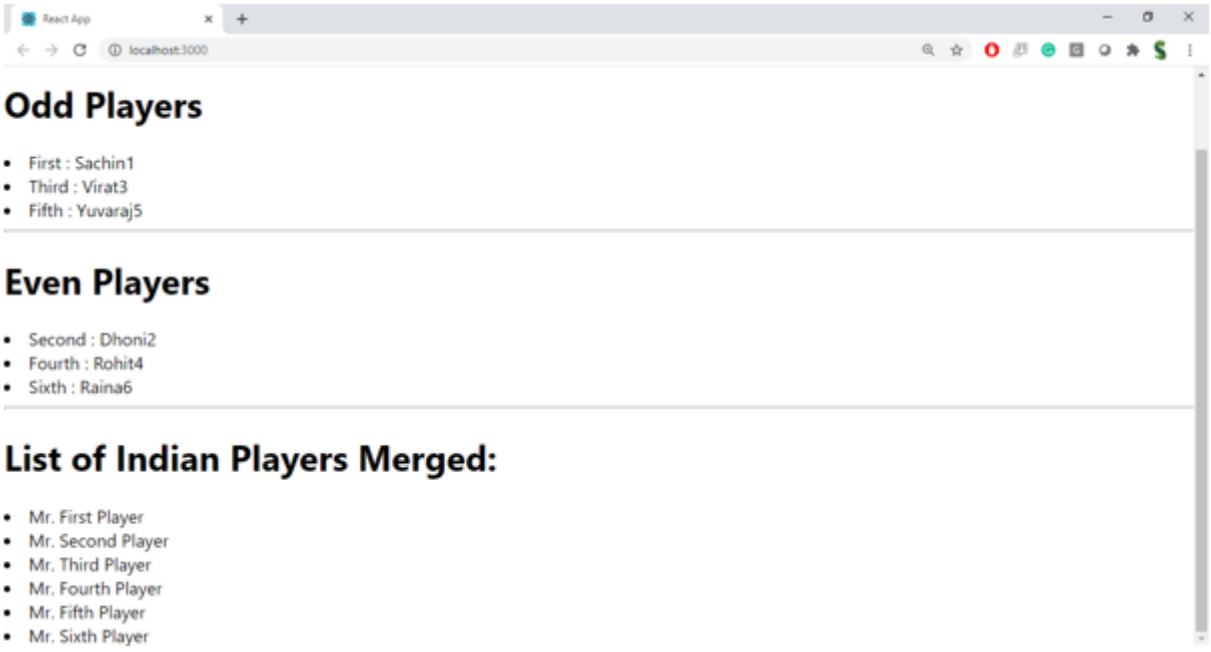
Display these two components in the same home page using a simple if else in the flag variable.

Output:

When Flag=true



When Flag=false



Hint:

```
var flag=true;
   if(flag===true)
   {
   return(
      <div>
         <h1> List of Players</h1>
          <ListofPlayers players={players}/>
          <hr/>
          <h1> List of Players having Scores Less than 70 </h1>
         <Scorebelow70 players={players}/>
      </div>
   )
   }
   else{
      return(
         <div>
         <div>
            <h1> Indian Team </h1>
            <h1> Odd Players </h1>
            {OddPlayers(IndianTeam)}
            <hr/>
            <h1> Even Players</h1>
            {EvenPlayers(IndianTeam)}
         </div>
         <hr/>
         <div>
            <h1> List of Indian Players Merged:</h1>
            <ListofIndianPlayers IndianPlayers={IndianPlayers}/>

         </div>
         </div>
      )
   }
```

SOLUTION:

ListofPlayers.js

```
import React from "react";
function ListofPlayers() {
   const players = [
      { name: "Virat Kohli", score: 95 },
      { name: "Rohit Sharma", score: 80 },
      { name: "Shubman Gill", score: 65 },
      { name: "KL Rahul", score: 60 },
      { name: "Hardik Pandya", score: 78 },
      { name: "Ravindra Jadeja", score: 55 },
      { name: "R Ashwin", score: 72 },
      { name: "Mohammed Shami", score: 40 },
      { name: "Jasprit Bumrah", score: 68 },
```

```
    { name: "Kuldeep Yadav", score: 74 },
    { name: "Mohammed Siraj", score: 50 }
  ];
  const lowScorePlayers = players.filter(player => player.score < 70);
  return (
    <div>
      <h2>List of Players</h2>
      <table border="1" cellPadding="8">
        <thead>
          <tr>
            <th>Name</th>
            <th>Score</th>
          </tr>
        </thead>
        <tbody>
          {
            players.map((player, index) => (
              <tr key={index}>
                <td>{player.name}</td>
                <td>{player.score}</td>
              </tr>
            ))
          }
        </tbody>
      </table>
      <br />
      <h2>Players with Score Below 70</h2>
      <ul>
        {
          lowScorePlayers.map((player, index) => (
            <li key={index}>
              {player.name} - {player.score}
            </li>
          ))
        }
      </ul>
    </div>
  );
}
export default ListofPlayers;
```

IndianPlayers.js

```
import React from "react";
function IndianPlayers() {
  const players = [
    "Virat",
    "Rohit",
```

```
    "Gill",
    "Rahul",
    "Hardik",
    "Jadeja"
  ];
  const [odd1, even1, odd2, even2, odd3, even3] = players;
  const T20players = [
    "Virat",
    "Rohit",
    "Hardik"
  ];
  const RanjiPlayers = [
    "Pujara",
    "Rahane",
    "Saha"
  ];
  const mergedPlayers = [...T20players, ...RanjiPlayers];
  return (
    <div>
      <h2>Odd Team Players</h2>
      <ul>
        <li>{odd1}</li>
        <li>{odd2}</li>
        <li>{odd3}</li>
      </ul>
      <h2>Even Team Players</h2>
      <ul>
        <li>{even1}</li>
        <li>{even2}</li>
        <li>{even3}</li>
      </ul>
      <h2>Merged Players</h2>
      <ul>
        {
          mergedPlayers.map((player, index) => (
            <li key={index}>{player}</li>
          ))
        }
      </ul>
    </div>
  );
}
export default IndianPlayers;
```

App.js

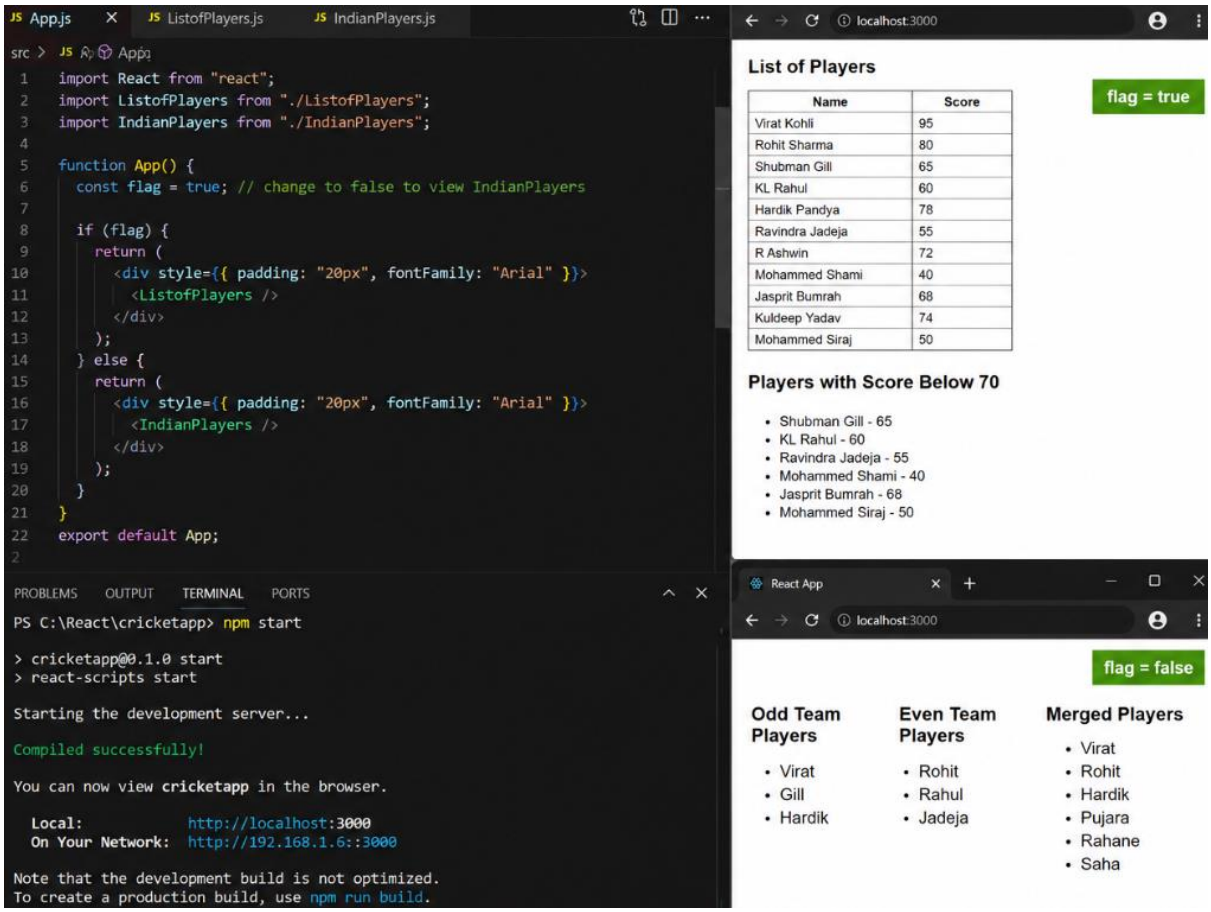
```
import React from "react";
import ListofPlayers from "./ListofPlayers";
import IndianPlayers from "./IndianPlayers";
```

```
function App() {
   const flag = true;
   if (flag) {
      return (
         <div>
            <ListofPlayers />
         </div>
      );
   } else {
      return (
         <div>
            <IndianPlayers />
         </div>
      );
   }
}
export default App;
```

RUN:

```
npm start
```

RESULT:



Objectives

- Define JSX
- Explain about ECMA Script
- Explain React.createElement()
- Explain how to create React nodes with JSX
- Define how to render JSX to DOM
- Explain how to use JavaScript expressions in JSX
- Explain how to use inline CSS in JSX

In this hands-on lab, you will learn how to:

- Use JSX syntax in React applications
- Use inline CSS in JSX

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **60 minutes.**

Create a React Application named “officespacerentalapp” which uses React JSX to create elements, attributes and renders DOM to display the page.

Create an element to display the heading of the page.

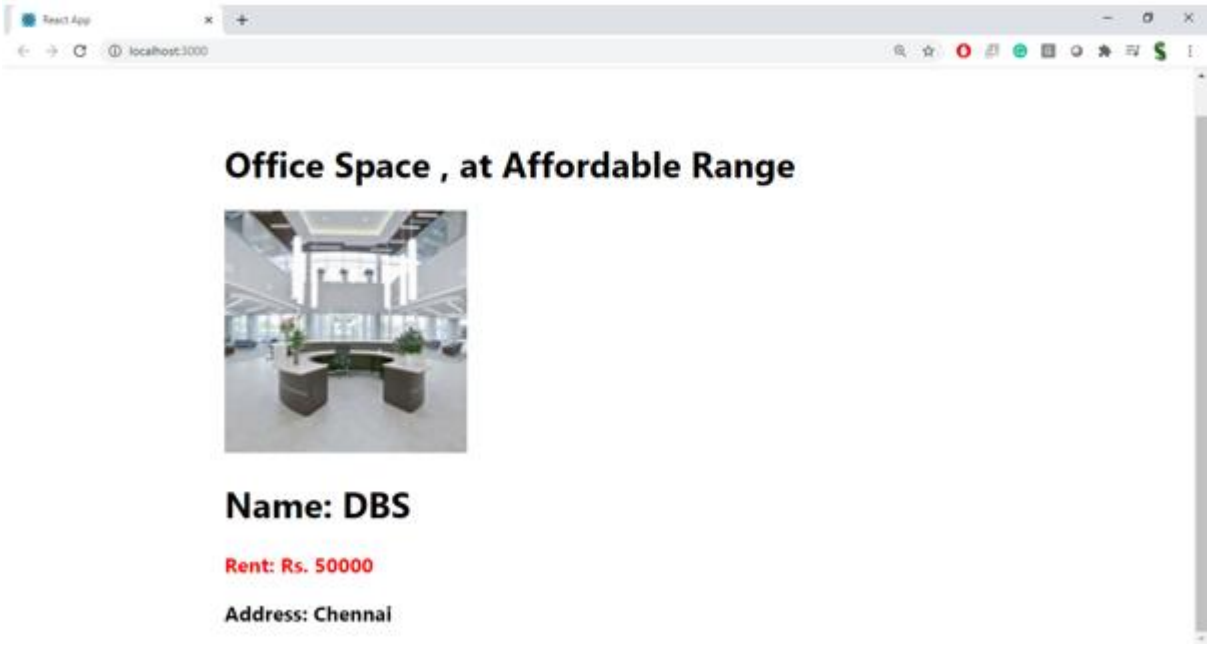
Attribute to display the image of the office space

Create an object of office to display the details like Name, Rent and Address.

Create a list of Object and loop through the office space item to display more data.

To apply Css, Display the color of the Rent in Red if it’s below 60000 and in Green if it’s above 60000.

Output:



Hint:

```
{
   let colors=[];
   if(ItemName.Rent<=60000)
   {
      colors.push('textRed');
   }
   else{
      colors.push('textGreen');
   }
```

```
const element="Office Space"
const jsxatt=<img src={sr} width="25%" height="25%" alt="Office Space"/>
const ItemName={Name:"DBS", Rent: 50000, Address:'Chennai'}
<h1>{element} , at Affordable Range </h1>
{jsxatt}
<h1>Name: {ItemName.Name}</h1>
<h3> Rent: Rs. {ItemName.Rent}</h3>
<h3> Address: {ItemName.Address}</h3>
```

SOLUTION:

App.js

```
import React from "react";
import "./App.css";
function App() {
  const officeSpaces = [
    {
      id: 1,
```

```
    name: "Tech Park",
    rent: 55000,
    address: "OMR, Chennai",
    image: "https://images.unsplash.com/photo-1497366754035-f200968a6e72?w=600"
  },
  {
    id: 2,
    name: "Business Hub",
    rent: 75000,
    address: "T Nagar, Chennai",
    image: "https://images.unsplash.com/photo-1497366412874-3415097a27e7?w=600"
  },
  {
    id: 3,
    name: "Corporate Tower",
    rent: 90000,
    address: "Guindy, Chennai",
    image: "https://images.unsplash.com/photo-1497366216548-37526070297c?w=600"
  }
];
return (
  <div className="App">
    <h1>Office Space Rental App</h1>
    {
      officeSpaces.map((office) => (
        <div className="card" key={office.id}>
          <img
            src={office.image}
            alt={office.name}
            width="350"
            height="220"
          />
          <h2>{office.name}</h2>
          <h3
            style={{
              color: office.rent < 60000 ? "red" : "green"
            }}
          >
            Rent : ₹{office.rent}
          </h3>
          <p>
            <b>Address :</b> {office.address}
          </p>
        </div>
      ))
    }
  </div>
);
```

```
}
export default App;
```

App.css

```
.App{
   text-align:center;
   font-family:Arial;
   background:#f2f2f2;
   padding:20px;
}
.card{
   display:inline-block;
   width:380px;
   margin:20px;
   padding:15px;
   background:white;
   border-radius:10px;
   box-shadow:0 0 10px gray;
}
img{
   border-radius:8px;
}
```

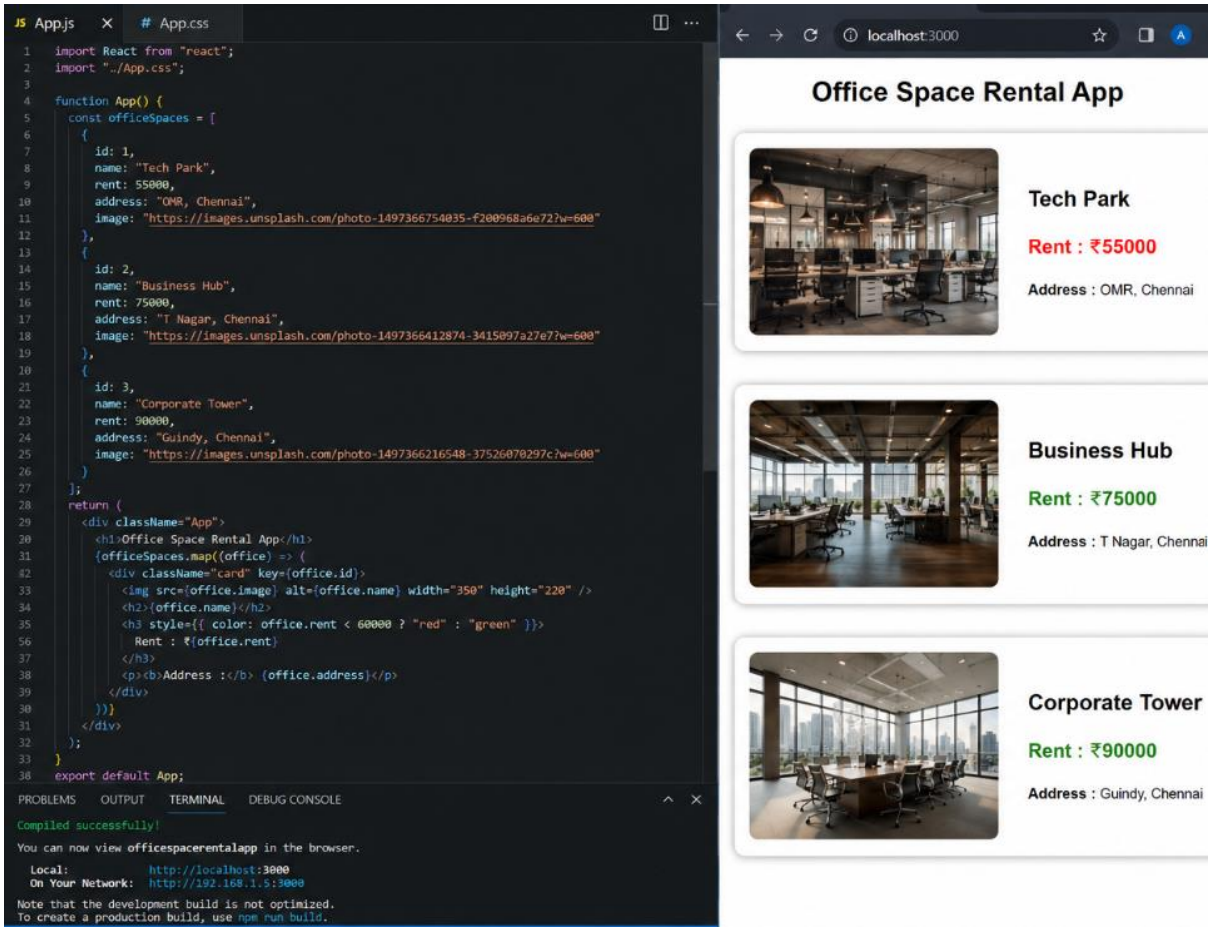
index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

RUN:

```
npm start
```

RESULT:



```
App.js
App.css

import React from "react";
import "../App.css";

function App() {
  const officeSpaces = [
    {
      id: 1,
      name: "Tech Park",
      rent: 55000,
      address: "OMR, Chennai",
      image: "https://images.unsplash.com/photo-1497366754035-f200968a6e72?w=600"
    },
    {
      id: 2,
      name: "Business Hub",
      rent: 75000,
      address: "T Nagar, Chennai",
      image: "https://images.unsplash.com/photo-1497366412874-3415097a27e7?w=600"
    },
    {
      id: 3,
      name: "Corporate Tower",
      rent: 90000,
      address: "Guindy, Chennai",
      image: "https://images.unsplash.com/photo-1497366216548-37526070297c?w=600"
    }
  ];
  return (
    <div className="App">
      <h1>Office Space Rental App</h1>
      {officeSpaces.map((office) => (
        <div className="card" key={office.id}>
          <img src={office.image} alt={office.name} width="350" height="220" />
          <h2>{office.name}</h2>
          <h3 style={{ color: office.rent < 60000 ? "red" : "green" }}>
            Rent : ₹{office.rent}
          </h3>
          <p><b>Address :</b> {office.address}</p>
        </div>
      ))}
    </div>
  );
}
export default App;

PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE

Compiled successfully!

You can now view officespacerentalapp in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.1.5:3000

Note that the development build is not optimized.
To create a production build, use npm run build.
```

Objectives

- Explain React events
- Explain about event handlers
- Define Synthetic event
- Identify React event naming convention

In this hands-on lab, you will learn how to:

- Implement Event handling concept in React applications
- Use this keyword
- Use synthetic event

Prerequisites

The following is required to complete this hands-on lab:

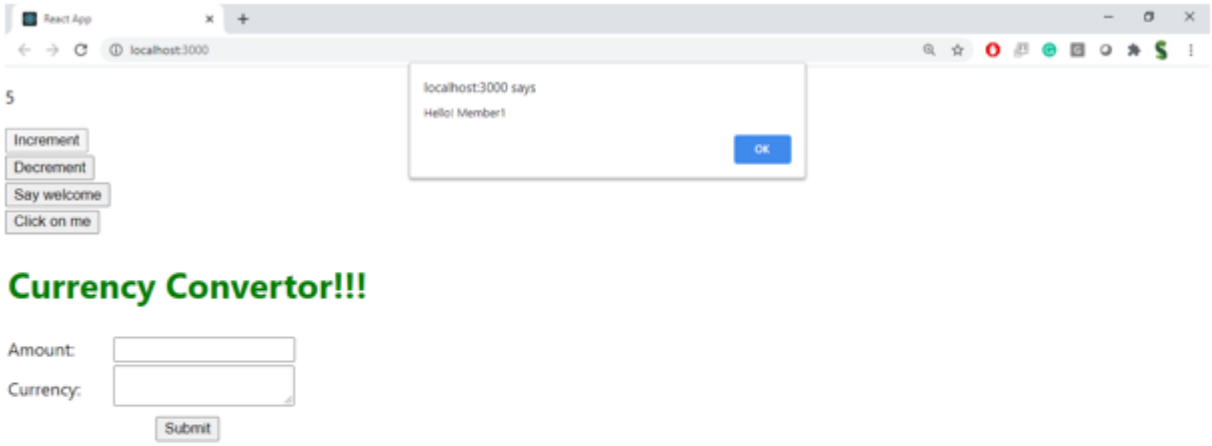
- Node.js
- NPM
- Visual Studio Code

Notes

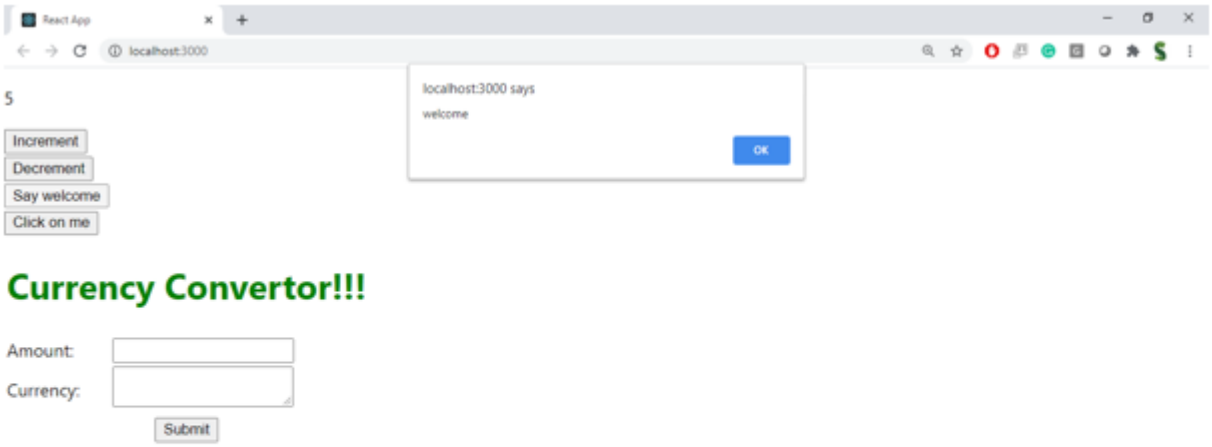
Estimated time to complete this lab: **90 minutes.**

Create a React Application “eventexamplesapp” to handle various events of the form elements in HTML.

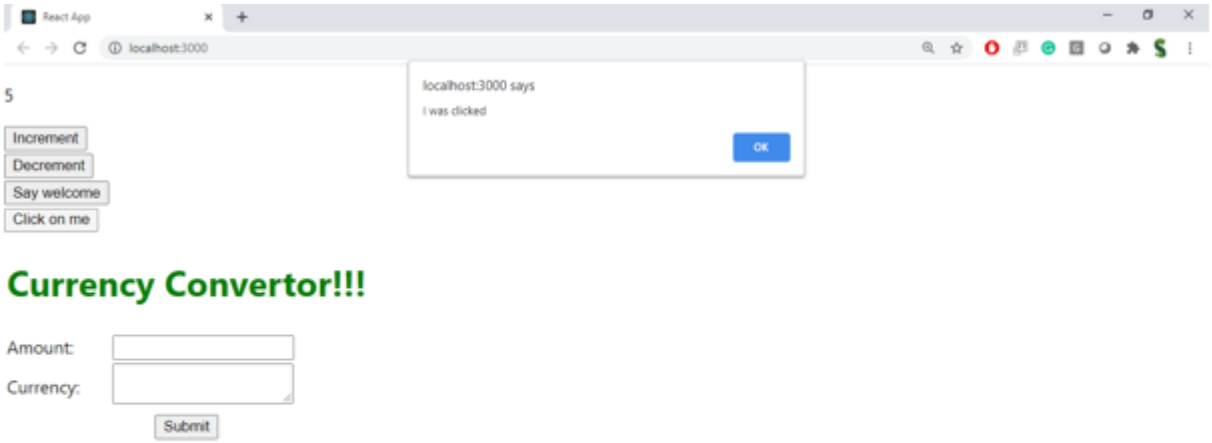
1. Create “Increment” button to increase the value of the counter and “Decrement” button to decrease the value of the counter. The “Increase” button should invoke multiple methods.
    a. To increment the value
    b. Say Hello followed by a static message.



2. Create a button “Say Welcome” which invokes the function which takes “welcome” as an argument.

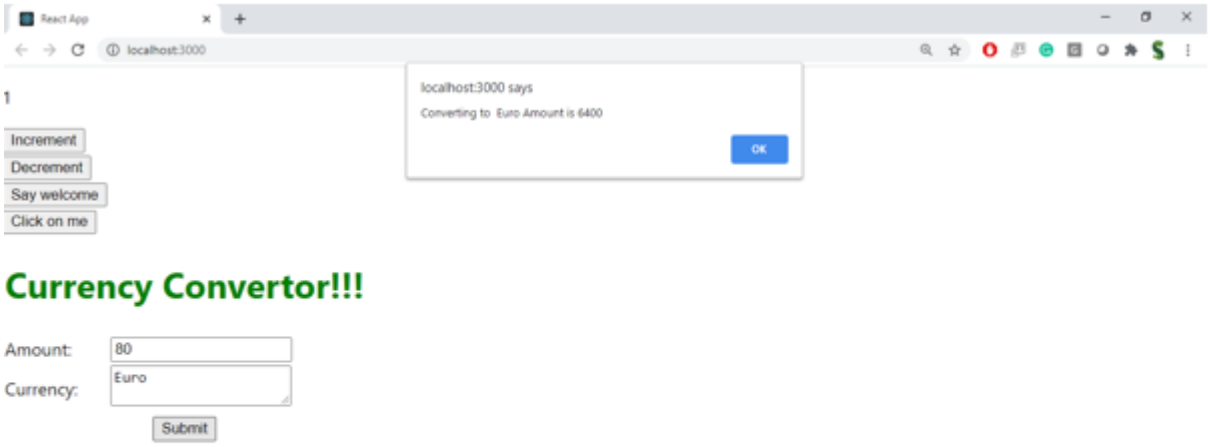


3. Create a button which invokes synthetic event “OnPress” which display “I was clicked”



Create a “CurrencyConvertor” component which will convert the Indian Rupees to Euro when the Convert button is clicked.

Handle the Click event of the button to invoke the handleSubmit event and handle the conversion of the euro to rupees.



SOLUTION:

App.js

```
import React, { Component } from "react";
import CurrencyConvertor from "./CurrencyConvertor";

class App extends Component {
  constructor() {
    super();
    this.state = {
      count: 0
    };
  }
  increment = () => {
    this.setState({
      count: this.state.count + 1
    });
  };
  decrement = () => {
    this.setState({
      count: this.state.count - 1
    });
  };
  sayHello = () => {
    alert("Hello! Welcome to React Events.");
  };
  sayWelcome = (msg) => {
    alert(msg);
  };
  handleClick = () => {
    alert("I was clicked");
  };
  handleIncrement = () => {
    this.increment();
    this.sayHello();
  };
  render() {
    return (
      <div style={{ textAlign: "center", marginTop: "30px" }}>
        <h1>React Event Examples</h1>
        <h2>Counter : {this.state.count}</h2>
        <button onClick={this.handleIncrement}>
          Increment
        </button>
         
        <button onClick={this.decrement}>
          Decrement
        </button>
        <br /><br />
```

```
      <button onClick={() => this.sayWelcome("Welcome")}>
        Say Welcome
      </button>
      <br /><br />
      <button onClick={this.handleClick}>
        OnPress
      </button>
      <hr />
      <CurrencyConvertor />
    </div>
  );
 }
}
export default App;
```

CurrencyConvertor.js

```
import React, { Component } from "react";
class CurrencyConvertor extends Component {
  constructor() {
    super();
    this.state = {
      rupees: "",
      euro: ""
    };
  }
  handleChange = (event) => {
    this.setState({
      rupees: event.target.value
    });
  };
  handleSubmit = () => {
    const euroValue =
      (parseFloat(this.state.rupees) / 90).toFixed(2);
    this.setState({
      euro: euroValue
    });
  };
  render() {
    return (
      <div>
        <h2>Currency Convertor</h2>
        <input
          type="number"
          placeholder="Enter Rupees"
          value={this.state.rupees}
          onChange={this.handleChange}
        />
```

```
             
            <button onClick={this.handleSubmit}>
              Convert
            </button>
            <h3>
              Euro : € {this.state.euro}
            </h3>
          </div>
      );
   }
}
export default CurrencyConvertor;
```

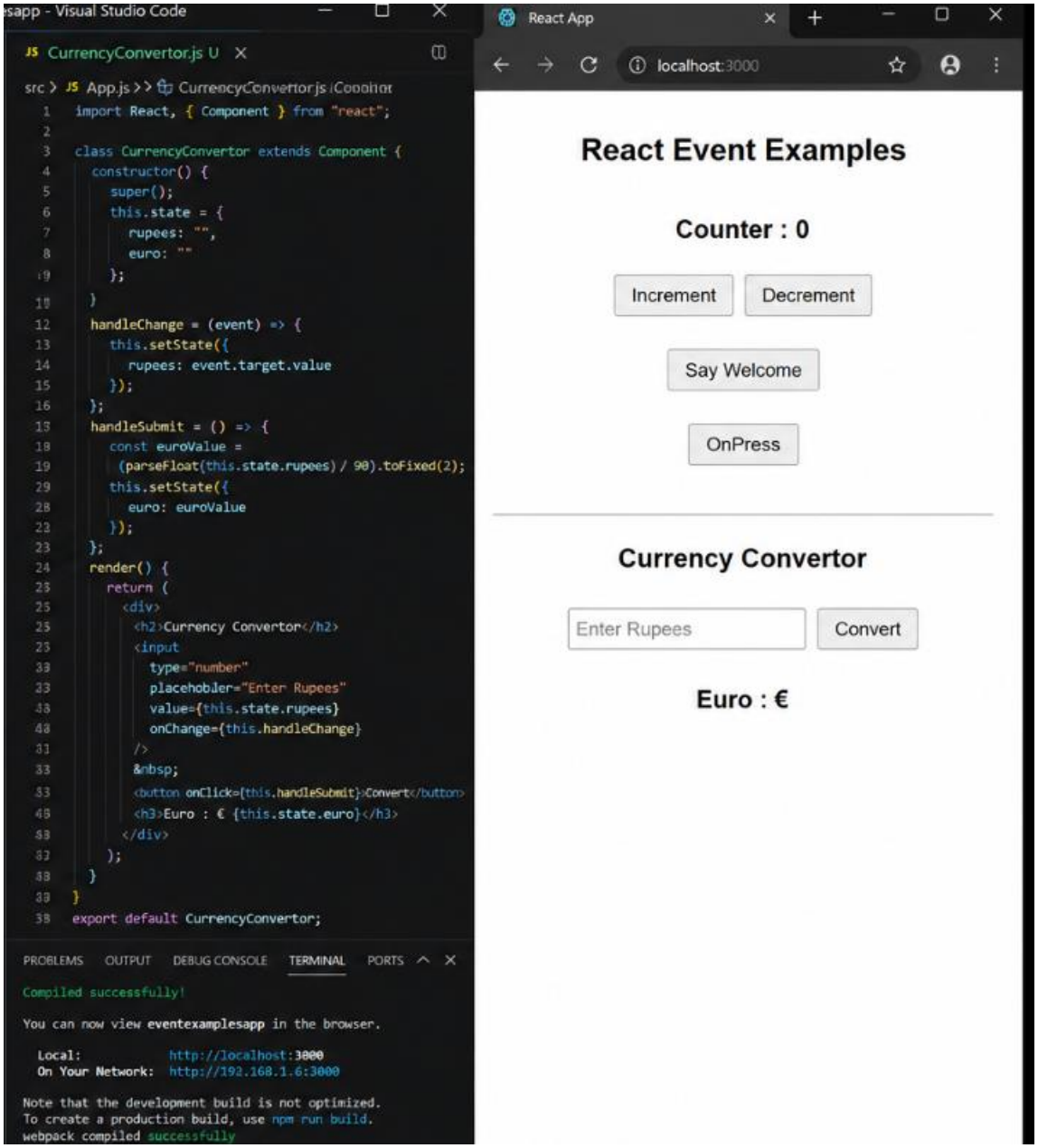
index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
const root = ReactDOM.createRoot(
  document.getElementById("root")
);
root.render(<App />);
```

RUN:

```
npm start
```

RESULT:



Objectives

- Explain about conditional rendering in React
- Define element variables
- Explain how to prevent components from rendering

In this hands-on lab, you will learn how to:

- Implement conditional rendering in React applications

Prerequisites

The following is required to complete this hands-on lab:

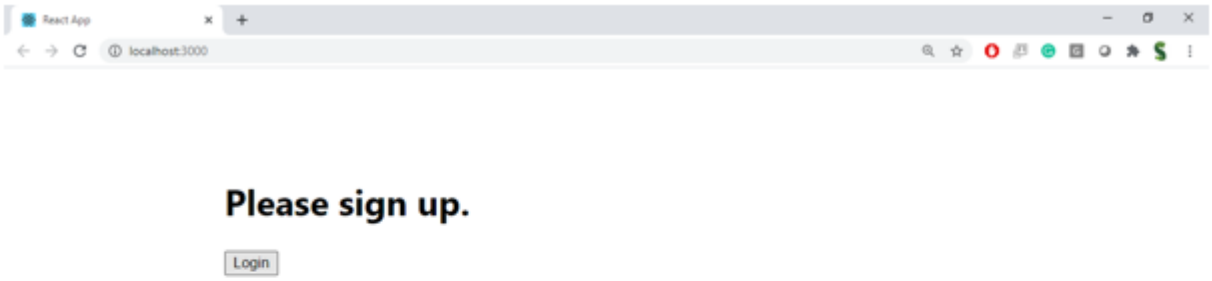
- Node.js
- NPM
- Visual Studio Code

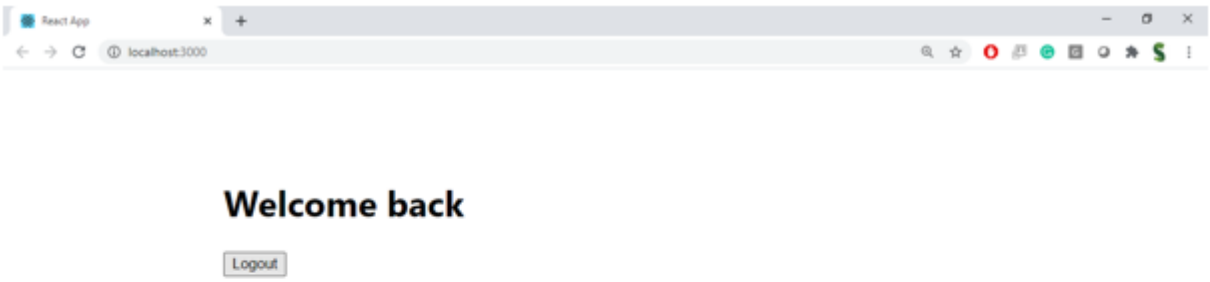
Notes

Estimated time to complete this lab: **60 minutes.**

Create a React Application named “ticketbookingapp” where the guest user can browse the page where the flight details are displayed whereas the logged in user only can book tickets.

The Login and Logout buttons should accordingly display different pages. Once the user is logged in the User page should be displayed. When the user clicks on Logout, the Guest page should be displayed.





Hint:

```
function LoginButton(props) {
  return (
    <button onClick={props.onClick}>
      Login
    </button>
  );
}
```

```
function LogoutButton(props) {
  return (
    <button onClick={props.onClick}>
      Logout
    </button>
  );
}
```

```
function Greeting(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return <UserGreeting />;
  }
  return <GuestGreeting />;
}
```

SOLUTION:

Guest.js

```
import React from "react";
function Guest() {
  return (
    <div>
      <h2>Welcome Guest</h2>
      <h3>Available Flights</h3>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>Flight</th>
            <th>From</th>
            <th>To</th>
            <th>Price</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>IndiGo</td>
            <td>Chennai</td>
            <td>Delhi</td>
            <td>₹5000</td>
          </tr>
          <tr>
            <td>Air India</td>
            <td>Chennai</td>
            <td>Mumbai</td>
            <td>₹4500</td>
          </tr>
          <tr>
            <td>SpiceJet</td>
            <td>Chennai</td>
            <td>Bangalore</td>
            <td>₹3000</td>
          </tr>
        </tbody>
      </table>
      <h3>Please Login to Book Tickets</h3>
    </div>
  );
}
export default Guest;
```

User.js

```
import React from "react";
function User() {
  return (
    <div>
```

```
        <h2>Welcome User</h2>
        <h3>Book Your Ticket</h3>
        <form>
          <p>
            Name :
            <input type="text" />
          </p>
          <p>
            Flight :
            <select>
              <option>IndiGo</option>
              <option>Air India</option>
              <option>SpiceJet</option>
            </select>
          </p>
          <button>
            Book Ticket
          </button>
        </form>
      </div>
    );
}
export default User;
```

App.js

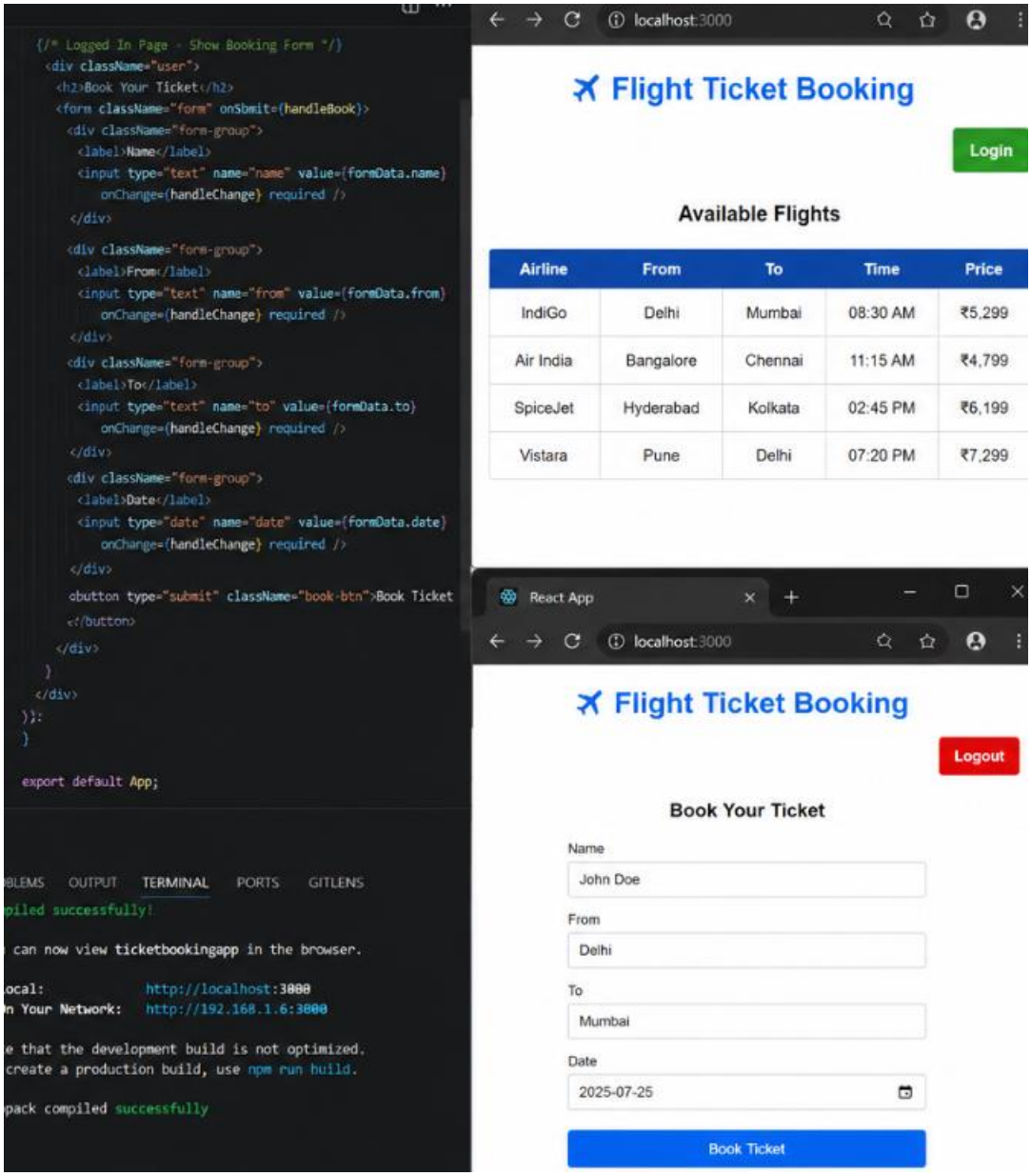
```
import React, { Component } from "react";
import Guest from "./Guest";
import User from "./User";
class App extends Component {
  constructor() {
    super();
    this.state = {
      isLoggedIn: false
    };
  }
  login = () => {
    this.setState({
      isLoggedIn: true
    });
  };
  logout = () => {
    this.setState({
      isLoggedIn: false
    });
  };
  render() {
    let button;
    let page;
```

```
      if (this.state.isLoggedIn) {
        button = (
          <button onClick={this.logout}>
            Logout
          </button>
        );
        page = <User />;
      }
      else {
        button = (
          <button onClick={this.login}>
            Login
          </button>
        );
        page = <Guest />;
      }
      return (
        <div
          style={{
            textAlign: "center",
            marginTop: "30px"
          }}
        >
          <h1>Flight Ticket Booking</h1>
          {button}
          <hr />
          {page}
        </div>
      );
    }
}
export default App;
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
const root = ReactDOM.createRoot(
  document.getElementById("root")
);
root.render(
  <App />
);
```

RESULT:



Objectives

- Explain various ways of conditional rendering
- Explain how to render multiple components
- Define list component
- Explain about keys in React applications
- Explain how to extract components with keys
- Explain React Map, map() function

In this hands-on lab, you will learn how to:

- Implement conditional rendering in React applications

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

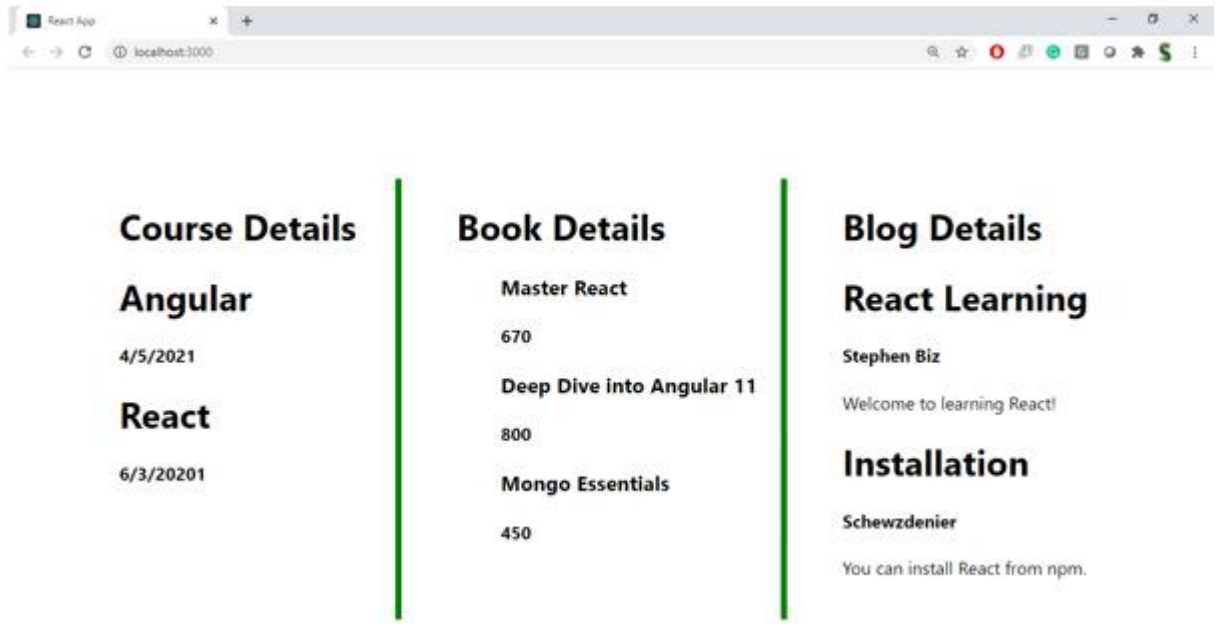
Notes

Estimated time to complete this lab: **60 minutes.**

Create a React App named “bloggerapp” in with 3 components.

1. Book Details
2. Blog Details
3. Course Details

Implement this with as many ways possible of Conditional Rendering.



Hint:

```
const bookdet = (
    <ul>
     {props.books.map((book) =>
      <div key={book.id}>
      <h3>  {book.bname}</h3>
        <h4>{book.price}</h4>
      </div>
     )}
    </ul>
  );
```

```
return (
    <div>
    <div>
    <div className="st2">
     <h1> Book Details</h1>
     {bookdet}
     </div>
     <div className="v1">
     <h1> Blog Details</h1>
     {content}
     </div>
     <div className="mystyle1">
      <h1> Course Details</h1>
     {coursedet}
    </div>
    </div>
    </div>
  );
 }
```

```
export const books=[
  {id:101,bname:'Master React',price:670},
  {id:102,bname:'Deep Dive into Angular 11 ',price:800},
  {id:103,bname:'Mongo Essentials',price:450},
];
```

SOLUTION:

BookDetails.js

```
import React from "react";
function BookDetails() {
   const books = [
      { id: 1, name: "React Guide", author: "Jordan Walke" },
      { id: 2, name: "Java Programming", author: "James Gosling" },
      { id: 3, name: "Python Basics", author: "Guido van Rossum" }
```

```
  ];
  return (
    <div>
      <h2>Book Details</h2>
      <ul>
        {
          books.map(book => (
            <li key={book.id}>
              <b>{book.name}</b> - {book.author}
            </li>
          ))
        }
      </ul>
    </div>
  );
}
export default BookDetails;
```

BlogDetails.js

```
import React from "react";
function BlogDetails() {
  const blogs = [
    {
      id: 1,
      title: "React Components",
      author: "John"
    },
    {
      id: 2,
      title: "Understanding JSX",
      author: "David"
    }
  ];
  return (
    <div>
      <h2>Blog Details</h2>
      {
        blogs.map(blog => (
          <div key={blog.id}>
            <h4>{blog.title}</h4>
            <p>Author : {blog.author}</p>
          </div>
        ))
      }
    </div>
  );
}
```

```
export default BlogDetails;
```

CourseDetails.js

```
import React from "react";
function CourseDetails() {
  const courses = [
    "React JS",
    "Spring Boot",
    "Java Full Stack"
  ];
  return (
    <div>
      <h2>Course Details</h2>
      <ol>
        {
          courses.map((course, index) => (
            <li key={index}>
              {course}
            </li>
          ))
        }
      </ol>
    </div>
  );
}
export default CourseDetails;
```

App.js

```
import React from "react";
import BookDetails from "./BookDetails";
import BlogDetails from "./BlogDetails";
import CourseDetails from "./CourseDetails";
function App() {
  const showBooks = true;
  const showBlogs = true;
  const showCourses = false;
  return (
    <div style={{ padding: "20px" }}>
      <h1>Blogger Application</h1>
      {/* Conditional Rendering using if */}
      {
        showBooks && <BookDetails />
      }
      {/* Conditional Rendering using Ternary */}
      {
        showBlogs
```

```
                ? <BlogDetails />
                : <h3>No Blogs Available</h3>
            }
            {/* Conditional Rendering using Element Variable */}
            {
               showCourses
                ? <CourseDetails />
                : <h3>Course Details Hidden</h3>
            }
        </div>
    );
}
export default App;
```

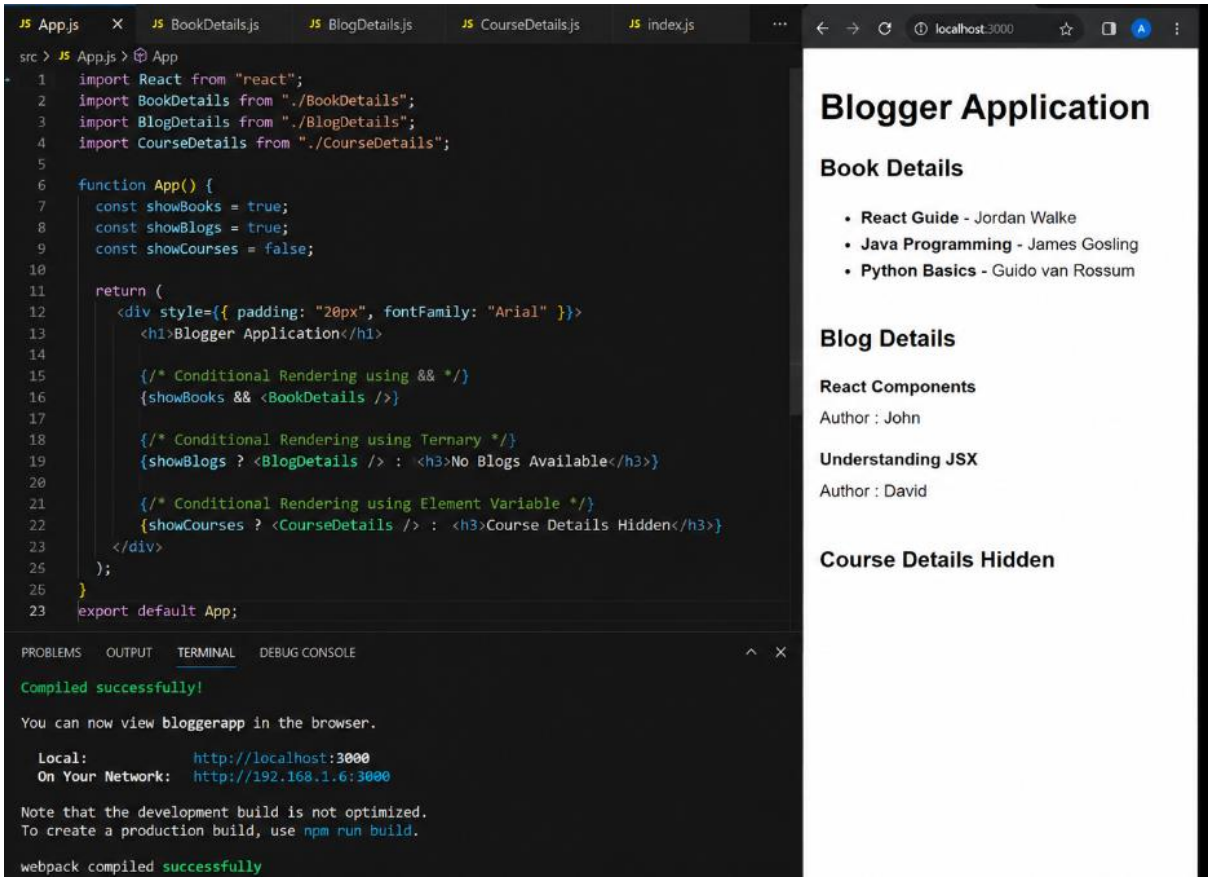
index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
const root = ReactDOM.createRoot(
   document.getElementById("root")
);
root.render(<App />);
```

RUN:

```
npm start
```

RESULT:


App.js
BookDetails.js
BlogDetails.js
CourseDetails.js
index.js
src > App.js > App
import React from "react";
import BookDetails from "./BookDetails";
import BlogDetails from "./BlogDetails";
import CourseDetails from "./CourseDetails";
function App() {
const showBooks = true;
const showBlogs = true;
const showCourses = false;
return (
<div style={{ padding: "20px", fontFamily: "Arial" }}>
<h1>Blogger Application</h1>
{/* Conditional Rendering using && */}
{showBooks && <BookDetails />}
{/* Conditional Rendering using Ternary */}
{showBlogs ? <BlogDetails /> : <h3>No Blogs Available</h3>}
{/* Conditional Rendering using Element Variable */}
{showCourses ? <CourseDetails /> : <h3>Course Details Hidden</h3>}

);
}
export default App;
PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
Compiled successfully!
You can now view bloggerapp in the browser.
Local: http://localhost:3000
On Your Network: http://192.168.1.6:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully
localhost:3000
Blogger Application
Book Details
React Guide - Jordan Walke
Java Programming - James Gosling
Python Basics - Guido van Rossum
Blog Details
React Components
Author : John
Understanding JSX
Author : David
Course Details Hidden